CLOUDSYS MASTER'S COURSE · PROJECT 4

Self-adaptive ML prediction of energy consumption

Project specification

Nabil Abdennadher Konstantin Skaburskas Mohsen Salimi
HES-SO / hepia, Geneva · Version 1.1, 6 October 2026

Purpose of this document. This specification fixes the scope, the data, the model, the architecture, the interfaces and the evaluation of Project 4, so that every student group builds the same system and can be assessed on the same basis. It replaces the earlier draft in which students had to design the forecasting model themselves: the model is now provided, and the project focuses on the cloud-edge architecture around it.

Where to find the material. Everything provided by the teaching team is in the course repository `github.com/salimi-khanghah/cloudsys-p4-students`: `data/` (Section 3), `kit/` (Section 4) and `spec/` (this document). It is a private repository: you get access through an invitation to your GitHub account. Clone it, never push to it, and build your system in your own repository.

Changes in version 1.1. (1) The low-performance rule compares the model with a simple reference, persistence, instead of a fixed threshold τ_m (Section 7.2). (2) The retraining trigger is $N = 8$ low-performance days, confirmed by the teaching team's dry run. (3) No Docker image or Dockerfile is provided: packaging the kit in containers is part of the project (Section 5.3). (4) The model kit, the command line of `train.py`, `meters.json` and the batch schema are described as delivered.

Contents

1 Context and objective 2
2 The self-adaptive loop 2
3 Data provided 3
4 Forecasting task and model kit 5
5 Architecture 8
6 Interfaces 10
7 Operating rules 11
8 Evaluation and tests 12
9 Deliverables, assessment and timeline 13
10 Roles and remaining actions of the teaching team 14
A Payload examples 15

1 Context and objective

The integration of intermittent, decentralised renewable sources and of new loads such as electric vehicles and heat pumps makes accurate forecasting of electricity consumption and production essential for grid operation. Smart meters provide the detailed, high-frequency data on which such forecasts are built. Following the principle of bringing the intelligence *close to the data*, forecasting is performed at the edge, near the meters, while the cloud is used to improve the forecasting model over time.

The objective of the project is to design, deploy and evaluate a **self-adaptive forecasting platform**: a machine learning model (MLM) runs on edge devices, monitors its own performance, and sends the data it fails on to the cloud; the cloud collects these data from several devices, retrains the model and redeploys the new version to the edge through the **Nuvla** edge-to-cloud platform.

What this course project is, and is not

- It **is** a cloud and edge systems project: containerisation, deployment and orchestration with Nuvla, edge-to-cloud communication, data storage, automated retraining, model versioning and roll-out, monitoring and resilience.
- It is **not** a forecasting research project. The forecasting model, its features and its training code are provided as a *model kit* (Section 4). Students run them; they do not have to design or tune them.

2 The self-adaptive loop

Figure 1 summarises the system. The loop runs once per (simulated) day.

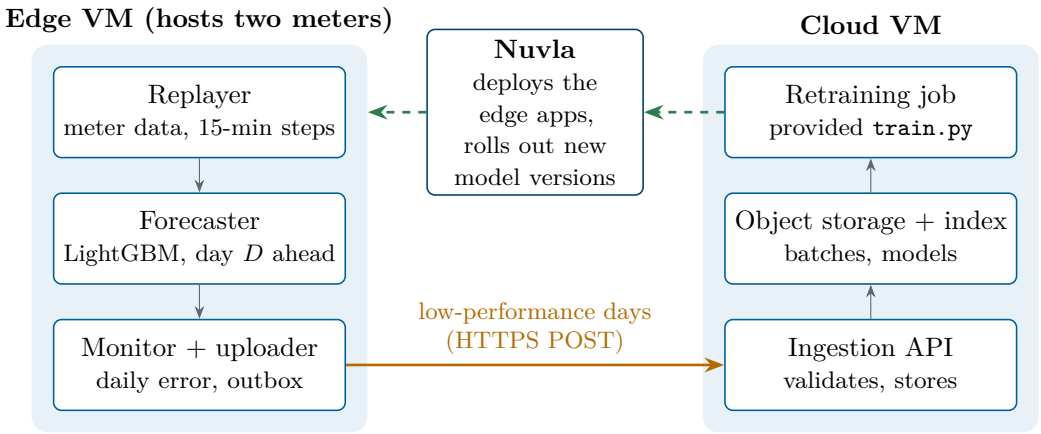


Figure 1: The self-adaptive loop. Edge VMs forecast and monitor; the cloud collects the days on which a forecast was poor, retrains, and Nuvla redeploys the new version.

1. **Initial model.** The teaching team provides two LightGBM models trained on the first part of the data (Section 3.6): one for consumers, one for prosumers.
2. **Deployment.** Students package the edge software, including the provided model kit, as Docker images and deploy it on the edge VMs with Nuvla.
3. **Forecasting.** Each edge VM replays its meters' data. At every simulated midnight it forecasts the 96 quarter-hours of the coming day D.

4. **Monitoring.** At the end of day D the actual values are known. The edge computes the daily error of its forecast and compares it with the error of persistence, a simple reference that repeats the values of one week earlier.
5. **Upload of low-performance days.** If the model did worse than persistence, the edge sends the day (forecast, actual values and the history needed to rebuild the inputs) to the cloud.
6. **Retraining.** When enough low-performance days have been collected for a category, the cloud retrains that category's model with the provided `train.py`.
7. **Roll-out and local acceptance.** A new version that passes the cloud-side check is rolled out by Nuvla. Each edge then decides locally whether to adopt it (Section 7.5). The loop continues with the next day.

Key principle. The edge VMs never train. All training happens in the cloud; the edge forecasts, monitors and decides whether to adopt a new version.

3 Data provided

3.1 Origin

The data come from the low-voltage grid of Marly (canton of Fribourg, Switzerland), operated by Groupe E and used as the testbed of the HES-SO flagship project SED (Smart Energy District). The grid is a radial tree fed from one 18 kV feed-in through 13 transformer stations. Students receive **10 smart meters located under transformer station 220 (LE COUCHANT)**. Five are *consumers* (they only draw power) and five are *prosumers* (they have photovoltaic panels and also export power). Each meter is the only meter of its connection point, so a file is the complete load of that point.

Role	Meter id	Node	Sub-branch	Class
C1	`27afc1ed`	183	207	consumer
C2	`bd21be01`	91	12	consumer
C3	`4c1e4d42`	202	207	consumer
C4	`8fd192c9`	146	264	consumer
C5	`6bedc031`	86	264	consumer
P1	`5c3492e5`	80	12	prosumer
P2	`f4e50206`	226	264	prosumer
P3	`f3544a98`	137	264	prosumer
P4	`9a685f1a`	182	207	prosumer
P5	`53229fee`	140	264	prosumer

Table 1: The ten meters. Node numbers refer to the 20 August 2026 release of the grid database. The sub-branch is the first node below station 220 on the path to the meter.

3.2 Files

The data folder is `data/` in the course repository:

```
meters/
  meter_27afc1ed.parquet        one file per meter (10 files)
  ...
weather_fribourg_15min.parquet  weather on the same time grid
```

```
meters.csv                    one summary line per meter
README.md                     short introduction
DATA_STRUCTURE.md             full description of the data
build.json                    how and when the folder was built
SHA256SUMS                    checksum of every file
```

All files are Apache Parquet (Zstandard compression). The cleaning (version 6.2) and every column are documented in `DATA_STRUCTURE.md`; the essentials follow.

3.3 Time grid

- **Period:** 7 October 2025 00:00 to 16 August 2026 23:45, i.e. **314 days** and **30 144 rows** in every file.
- **Resolution:** 15-minute intervals, labelled by their *start*; every value is an average over its interval.
- **Time zone:** `timestamp` is in Swiss local time (Europe/Zurich), stored without time zone; the same instant in UTC is in `timestamp_utc`.
- **No missing rows:** every day has 96 rows. A missing measurement is an empty value, never a missing row. Day d ($0 = 7$ October 2025), slot s ($0 = 00{:}00$) is row $96d + s$ in every file, so files join one-to-one on `timestamp`.
- **Clock changes:** the doubled autumn hour (26 October 2025) is folded into single slots; the non-existent spring hour (29 March 2026) is kept as four slots so that every day has 96 rows, and `timestamp_utc` is empty on those four slots.

3.4 Meter files

Column	Unit	Meaning
`timestamp, timestamp_utc`		start of the 15-minute interval (local, UTC)
`ap_kw`	kW	net active power; positive = drawn from the grid, negative = sent to the grid
`rp_kvar`	kVAR	net reactive power
`van_v, vbn_v, vcn_v`	V	phase-to-neutral voltage of phases A, B, C
`ian_a, ibn_a, icn_a`	A	phase current of phases A, B, C (signed)
`<channel>_origin`	code	quality code of each value (one column per measurement)

Table 2: The 18 columns of a meter file. Only `ap_kw` is forecast in this project.

Quality codes. Every value carries an `_origin` code. Three rules are enough:

Rule	Meaning
`origin < 10`	measured
`origin < 20`	measured or lightly repaired: **safe to train on and to evaluate on**
`origin >= 90`	missing: the value is empty (NaN)

Over the ten meters, 97.9 % of active-power values are measured, 0.6 % interpolated and 1.4 % missing. Each file also carries metadata (`meter_id`, `meter_class`, `node_id`, `cleaning_version`, the code list and per-channel counts).

3.5 Weather file

`weather_fribourg_15min.parquet` comes from the MeteoSwiss station Grangeneuve (Fribourg) and has the same `timestamp` column as the meter files. Its nine variables are air temperature (`temp_air_2m_c`, °C), relative humidity (`humidity_2m_pct`, %), dew point (`dew_point_2m_c`, °C), station pressure (`pressure_qfe_hpa`, hPa), wind speed (`wind_speed_ms`, m/s), wind direction (`wind_direction_deg`, °), precipitation (`precipitation_mm`, mm per 15 min), global radiation (`global_radiation_wm2`, W/m^2) and sunshine (`sunshine_min`, minutes per 15 min). The source 10-minute values were shifted to the start of their interval, converted to local time and averaged onto the 15-minute grid. The file has no origin codes and is complete except the four spring clock-change slots.

3.6 Split of the period

The provided models are trained on the first part of the period only. The remaining part is replayed on the edge, where the models meet conditions they have not seen (rising solar production in spring and summer, falling consumption), so that performance degrades and the retraining loop is exercised.

Part	From	To	Days
A1: training of the provided models	7 Oct 2025	3 Mar 2026	148
A2: validation, release check	4 Mar 2026	31 Mar 2026	28
B: replay on the edge	1 Apr 2026	16 Aug 2026	138
Total			314

Table 3: Chronological split. Part B is never used to build the initial models or μ_m.

The edge needs 14 days of history before the first forecast (Section 7.5); the replayer therefore starts on 18 March 2026 and the first forecast is issued for 1 April 2026.

4 Forecasting task and model kit

4.1 Task

- **Target:** net active power `ap_kw` of one meter.
- **Horizon:** day-ahead, the 96 quarter-hours of day D.
- **Issue time:** midnight at the start of day D; the load is known up to the end of day $D-1$.
- **Models:** one LightGBM model per category: f_{cons} for C1 to C5 and f_{pros} for P1 to P5. The meter identity is an input feature.

LightGBM predicts one value per row, so one row corresponds to one target slot s of one day D, and a day-ahead forecast is obtained by evaluating the model on the 96 rows of day D:

$$\hat{y}_{m,D,s} \;=\; \mu_m \cdot f_{c(m)}(\mathbf{x}_{m,D,s})\,, \qquad s = 0, \ldots, 95, \tag{1}$$

where $c(m) \in \{\text{cons}, \text{pros}\}$ is the category of meter m and μ_m its scale.

Per-meter normalisation. Meters differ in size, so the model works on normalised load $\tilde{y}_{m,t} = y_{m,t}/\mu_m$, with

$$\mu_m \;=\; \frac{1}{|\mathcal{T}_{A1}|} \sum_{t \in \mathcal{T}_{A1}} |y_{m,t}| \,, \tag{2}$$

the mean absolute load of meter m over part A1 (valid values only). The absolute value is used because the net load of a prosumer can average close to zero. μ_m is fixed and shipped in the model kit.

Features. The feature vector $\mathbf{x}_{m,D,s}$ is built by the provided `p4kit/features.py`:

Group	Features	Source
Past load	$\tilde{y}_{m,D-1,s}$, $\tilde{y}_{m,D-2,s}$, $\tilde{y}_{m,D-7,s}$ (same slot 1, 2 and 7 days earlier); mean of $\tilde{y}_{m,D-1,\cdot}$	meter file
Calendar	slot of the day s; day of the week; weekend flag; public holiday flag (canton of Fribourg)	timestamp
Weather	global radiation, air temperature and sunshine of day D at slot s	weather file
Identity	meter id (categorical)	metadata

Table 4: Inputs of the models.

A training row is kept only if its target and all its load inputs have `origin < 20`. At inference no slot is dropped: invalid inputs are passed to the model as missing values, so a forecast always has 96 values. Public holidays come from the Python package `holidays` (Switzerland, canton of Fribourg); the dates used are listed in `MODEL_CARD.md`.

4.2 The model kit

The model kit is the folder `kit/` of the course repository. Students may read it but must not change it; any change would make results incomparable between groups.

Item	Content and purpose
`models/`	`model_cons_v0001.txt` and `model_pros_v0001.txt`, the two initial LightGBM models (native text format), trained on part A1
`p4kit/features.py`	builds $\mathbf{x}_{m,D,s}$ from raw data; **the same function is used for training and inference**
`p4kit/predict.py`	`predict_day()` and a command line: the 96 forecasts of day D for one meter, in kW
`p4kit/train.py`	command-line training job: data in, model, metrics and release decision out (Section 4.3)
`p4kit/metrics.py`	the daily MAE and the low-performance rule (Section 7.2)
`p4kit/verify.py`	checks a deployment against the test vectors (test F1)
`p4kit/io.py`	reads the data folder, the weather file and low-performance batches
`meters.json`	per meter: category, node and scale μ_m; the split dates; the parameters of the loop (block `loop`)
`requirements.txt`	the exact version of every package, for **Python 3.11**
`test_vectors/`	30 reference inputs (10 meters × 3 days of part B) with their 96 expected outputs
`tests/`	unit tests of the kit (pytest)
`README.md, MODEL_CARD.md`	how to use the kit; what the model does, its inputs and its known limits

Table 5: Contents of the model kit. No Dockerfile is provided (Section 5.3).

4.3 The retraining job

`train.py` is a non-interactive command, run in a container in the cloud:

```
python -m p4kit.train --category pros \
    --data-dir /data \
    --weather /data/weather_fribourg_15min.parquet \
    --meters meters.json \
    --extra-days /data/lowperf/pros \
    --current-model /models/pros/v0003/model.txt \
    --out /models/pros/v0004 \
    --seed 42
```

`-data-dir` is the provided data folder, from which `train.py` reads parts A1 and A2 only; `-extra-days` is a folder of low-performance batches (any depth, JSON or Parquet, duplicates read once); `-current-model` is the version currently released. The job trains a new model of the given category on part A1 *plus* the low-performance days received so far for that category. Keeping part A1 in every retraining prevents the model from forgetting normal days while it learns the difficult ones. The chronologically last 20 % of the low-performance days (at least one) are held out for the release check. The number of trees is fixed (no early stopping) and read from `meters.json`.

It writes `model.txt` and `metrics.json` into `-out`. `metrics.json` holds the errors of the new model and of the current one on part A2 and on the held-out days, the batches used and held out (lineage), the training time, and the release decision (`"release": {"released": true|false, "reason": ...}`, Section 7.4). It never overwrites an existing `model.txt`. A training takes about 10 s on one CPU.

5 Architecture

5.1 Infrastructure

Machine	Number	Role and size
Edge VM	5	hosts two meters each (Table 7); **fixed size: 1 vCPU, 2 GB RAM**, Linux with Docker and NuvlaEdge
Cloud VM	1	ingestion API, storage, registry and retraining job; 2 to 4 vCPU, 8 GB RAM
Nuvla	service	`nuvla.io` (or the instance provided by the Nuvla team), one organisation per group

Table 6: Infrastructure per student group. All VMs are created on the course cloud (SWITCHengines by default). Placing the edge VMs and the cloud VM in different regions is encouraged, to obtain a real network separation.

The edge VM size is a constraint, not a suggestion: it keeps the edge an edge. The edge pipeline must run within it.

Edge VM	Consumer	Prosumer
edge-1	C1	P1
edge-2	C2	P2
edge-3	C3	P3
edge-4	C4	P4
edge-5	C5	P5

Table 7: Assignment of meters to edge VMs. Each VM serves one consumer and one prosumer, so every VM is affected by a roll-out of either category.

A **device** in this document is one meter served by an edge VM: a VM hosts two devices, each with its own replayer, forecaster and monitor instance and its own `DEVICE_ID` (the meter id).

5.2 Containers

Everything runs in Docker containers. NuvlaEdge itself runs as containers on each edge VM, and every application deployed by Nuvla is described by a Docker Compose file.

Where	Container	Responsibility	Deployed by
Edge VM	NuvlaEdge	agent, job engine, MQTT data gateway, VPN client	NuvlaEdge installer
	replayer	reads a meter file and publishes one 15-minute reading per tick	Nuvla
	forecaster	keeps 14 days of history, forecasts day D at midnight with `predict.py`	Nuvla
	monitor	computes the daily error, flags low-performance days, keeps the outbox, uploads	Nuvla
Cloud VM	api	ingestion API (FastAPI): validates and stores batches, serves model files	Docker Compose
	trainer	runs `train.py` on trigger, evaluates, registers, requests roll-out	Docker Compose
	mlflow	model registry and experiment tracking	Docker Compose
	postgres	index of batches, metrics and model versions	Docker Compose
	object store	S3 bucket (SWITCHengines S3, or MinIO in a container)	Docker Compose

Table 8: Containers. One edge image may provide the three edge roles through different commands.

5.3 Packaging the model kit

No Docker image or Dockerfile is provided: writing the images is part of the project. Every image that runs the kit (the forecaster and monitor on the edge, the trainer in the cloud) must respect three rules.

1. **Python 3.11, and exactly `kit/requirements.txt`.** Start from a Python 3.11 base image and install that file as it is, with no other versions.
2. **Reproduce the test vectors (test F1).** Inside the image, run from the kit folder

   ```
   python -m p4kit.verify --vectors test_vectors/
   ```

   It prints `PASS` or `FAIL` and the largest deviation for each of the 30 files, and exits with code 0 only if all pass within 10^{-6} kW. A failure means the environment differs from the reference: check the Python and package versions first.
3. **Build for linux/amd64.** The edge VMs are x86-64. On Apple Silicon or another ARM machine, build with `docker build --platform linux/amd64`.

For reference, the teaching team measured `predict.py` at 1 vCPU and 2 GB: about 15 ms per forecast and 165 MB of memory, after a start-up of a few seconds per process.

5.4 Network

- **Control plane:** each NuvlaEdge agent talks to Nuvla over outbound HTTPS (port 443).
- **Data plane:** the monitor talks to the ingestion API over outbound HTTPS. The edge downloads model files from the API, so no storage credentials are needed on the edge.

- **On the edge VM:** replayer, forecaster and monitor exchange data through the NuvlaEdge data gateway (MQTT); this traffic never leaves the VM.
- Nothing connects *into* an edge VM. If the link to the cloud is lost, the edge keeps forecasting and stores its uploads in a local outbox.

6 Interfaces

6.1 Edge: MQTT topics

Topic	Publisher	Payload
`meters/<id>/readings`	replayer	one 15-minute reading (example in Appendix A.1)
`meters/<id>/forecast`	forecaster	the 96 forecasts of day D, with the model version
`meters/<id>/daily`	monitor	daily error, error of persistence and flag of day D
`replay/clock`	replayer	current simulated time, for synchronisation

6.2 Cloud: ingestion API

Endpoint	Behaviour
`POST /v1/lowperf`	one low-performance day (schema in Appendix A.2); `201` if stored, `200` if the `batch_id` was already stored, `422` if the schema is invalid
`POST /v1/metrics`	daily error of one device (sent every day, flagged or not)
`GET /v1/models/{category}/{version}`	returns the model file of that version
`GET /v1/models/{category}/latest`	returns the version number currently released
`GET /v1/health`	liveness check

Every request carries a per-device bearer token. The API validates every batch before storing it: required fields, array lengths (96 values per day), timestamps in order, a known `device_id`, finite values within physical bounds.

6.3 Storage layout

```
s3://p4-<group>/
  lowperf/category=<cons|pros>/device=<id>/day=<YYYY-MM-DD>/<batch_id>.parquet
  models/<cons|pros>/v<NNNN>/model.txt
  models/<cons|pros>/v<NNNN>/metrics.json
  base/                                  (copy of the provided data folder)
```

A batch is posted as JSON and may be stored as JSON or as a one-row Parquet file with the same fields; `train.py` reads both. The trainer copies or mounts the batches of a category into one folder and passes it as `-extra-days`.

PostgreSQL holds one row per batch, one row per daily metric and one row per model version, including the list of batches each version was trained on (lineage).

6.4 Nuvla applications and parameters

The edge software is registered in Nuvla as one application (Docker Compose) and deployed to the five edge VMs as one **deployment group**. Its parameters are:

Parameter	Meaning
`DEVICE_IDS`	the two meter ids served by this VM
`API_URL, API_TOKEN`	address of and credential for the ingestion API
`MODEL_VERSION_CONS, MODEL_VERSION_PROS`	model versions released for each category
`REPLAY_START, REPLAY_SPEED`	wall-clock start and simulated seconds per real second

A roll-out is a change of `MODEL_VERSION_<CAT>` in the deployment group, made by the trainer through the Nuvla REST API (Python client `nuvla-api`). History and outbox are stored in Docker volumes, so they survive the restart caused by a redeployment.

7 Operating rules

7.1 Replay

The replayer publishes the readings of its meter in simulated time. The default speed is **one simulated day per real minute** (`REPLAY_SPEED = 1440`): part B (138 days) replays in about 2.3 hours, plus 14 minutes of warm-up. All edge VMs of a group start at the same `REPLAY_START`.

7.2 Daily error and low-performance rule

At the end of day D the monitor computes, for device m, the daily error that is reported every day,

$$\mathrm{MAE}_{m,D} \;=\; \frac{1}{|\mathcal{S}_{m,D}|} \sum_{s \in \mathcal{S}_{m,D}} |y_{m,D,s} - \hat{y}_{m,D,s}|\,, \tag{3}$$

where $\mathcal{S}_{m,D}$ is the set of slots whose actual value has `origin < 20`. A day with fewer than 90 valid slots is not evaluated.

To decide whether the day was a *low-performance* day, the forecast is compared with **persistence**, a simple reference that forecasts each quarter-hour with the same quarter-hour one week earlier, $y_{m,D-7,s}$. Both errors are computed over the same slots $\mathcal{S}'_{m,D}$, those where the actual value *and* the value of $D-7$ have `origin < 20`:

$$\mathrm{MAE}^{\mathrm{model}}_{m,D} = \frac{1}{|\mathcal{S}'_{m,D}|} \sum_{s \in \mathcal{S}'_{m,D}} |y_{m,D,s} - \hat{y}_{m,D,s}|\,, \qquad \mathrm{MAE}^{\mathrm{pers}}_{m,D} = \frac{1}{|\mathcal{S}'_{m,D}|} \sum_{s \in \mathcal{S}'_{m,D}} |y_{m,D,s} - y_{m,D-7,s}|\,. \tag{4}$$

The day is evaluated only if $|\mathcal{S}'_{m,D}| \geq 90$, and it is **flagged as low performance** if

$$\mathrm{MAE}^{\mathrm{model}}_{m,D} \;>\; (1 + m_{\mathrm{flag}})\, \mathrm{MAE}^{\mathrm{pers}}_{m,D}, \qquad m_{\mathrm{flag}} = 0, \tag{5}$$

that is, if the model did worse than simply repeating last week. The margin m_{flag} and the other parameters of the loop are in the block `loop` of `meters.json`; the rule is implemented in `p4kit.metrics.low_performance`. A comparison with persistence, rather than a fixed threshold in kW, keeps working when the level of the load changes with the seasons; the teaching team's dry run found that a fixed threshold set in March flagged almost no day after May.

A flagged day is uploaded with the 96 forecasts, the 96 actual values and origin codes of day D, and the actual values and origin codes of days $D-7$ to $D-1$ (Appendix A.2), so that the cloud can rebuild every input of that day.

7.3 Retraining trigger

Retraining is decided per category. The trainer starts a new training of category c when, since the last training of that category,

- at least $N = 8$ flagged days have been received, **from at least two different devices** of the category, so that one faulty meter cannot force a retrain; or
- 14 simulated days have passed and at least one flagged day has been received (fallback).

These values were chosen by the teaching team's dry run: over part B they give about 15 % (consumers) and 11 % (prosumers) flagged days and about a dozen trainings per category.

7.4 Release in the cloud

The models are compared on the **pooled nMAE** of the category: the mean of $|y - \hat{y}|/\mu_m$ over all valid slots of all its devices, so that a large meter does not dominate. A trained challenger is **released** (registered as the latest version and rolled out through Nuvla) only if, compared with the current version of the category,

- its pooled nMAE on part A2 does not increase by more than 5 %, and
- its pooled nMAE on the held-out low-performance days is lower.

`train.py` applies this rule and writes the decision in `metrics.json`. Otherwise the version is kept in the registry, marked as rejected, and nothing is rolled out.

7.5 Local acceptance on the edge

One model serves five different meters, so a release that helps most of them may harm one. After a roll-out, each device therefore evaluates the old and the new version on its own last 7 complete days (it holds 14 days of history, enough to rebuild the inputs of those 7 days) and **adopts the new version only if its MAE over these 7 days is not higher**. The decision is reported in the daily metrics. A device that rejects a version keeps forecasting with its current one and re-evaluates at the next release.

8 Evaluation and tests

8.1 Prequential evaluation and baseline

Part B is not a static test set once retraining starts. Evaluation is therefore *prequential*: every day is forecast with the model in service at that moment, before its actual values are known, and scored once they arrive. Each group also runs a **no-retraining baseline**: the same replay with the initial models frozen.

8.2 Metrics to report

Aspect	Metrics
Forecast quality	daily MAE and nMAE (MAE divided by μ_m) per device, with and without retraining, and of persistence as a reference; their mean over part B
Adaptation	number of flagged days, retrainings, releases and local rejections; model version in service per device over time
Latency	time from the trigger to the new version being active on all devices that adopt it
Cost	bytes sent edge to cloud per day and in total; CPU and memory use of the edge VM
Resilience	behaviour and data loss in each test of Section 8.3

The central result is one figure: the daily nMAE over part B with and without retraining, averaged over the devices of each category.

8.3 Functional and resilience tests

Test	Action	Expected behaviour
F1 Deployment correctness	run `python -m p4kit.verify --vectors test_vectors/` inside every image that runs the kit	`PASS` for all 30 files (tolerance 10^{-6} kW), exit code 0
F2 End-to-end loop	full replay of part B	at least one retraining and one release per category
F3 Schema validation	send a malformed batch	`422`, nothing stored
F4 Idempotence	send the same batch twice	stored once, second answer `200`
R1 Network cut	disconnect one edge VM for 10 simulated days	forecasting continues; outbox delivered on reconnection; no batch lost or duplicated
R2 API down	stop the API container for 5 simulated days	same as R1 for all edges
R3 Container crash	kill the forecaster during the replay	restarted automatically; history recovered from the volume; at most one forecast missed
R4 Redeployment	roll out a new version during the replay	no reading lost; history preserved
R5 Bad release	roll out a deliberately worse model	devices reject it by local acceptance

Table 9: Minimum test set. Groups may add tests.

9 Deliverables, assessment and timeline

9.1 Deliverables

1. Requirements and architecture documentation (and presentation), including the justification of technology choices.
2. The forecasting service deployed on the edge VMs through Nuvla.
3. The cloud retraining pipeline: ingestion API, storage, retraining job, registry, release and roll-out.

4. The integrated application (edge + cloud) orchestrated by Nuvla, with the full replay of part B, with and without retraining.
5. A test report covering the functional and resilience tests of Section 8.3 and the metrics of Section 8.

9.2 Assessment

As announced in the project list: requirements analysis and documentation quality (15 %); architecture and technology choice justification (30 %); pipeline implementation (30 %); testing rigour (15 %). Forecast accuracy itself is **not** assessed; the comparison with and without retraining is assessed as part of the pipeline and the tests.

The four weights of the project list add up to 90 %. The remaining 10 % is to be confirmed by the course responsible.

9.3 Timeline

Week	Dates	Milestone
0	Tue 6 Oct 2026, 8:45	project selection; kick-off; course repository (data, model kit, specification) shared
1	6 to 13 Oct	architecture document draft; accounts (cloud, Nuvla); data explored; one edge VM enrolled in Nuvla
2	13 to 20 Oct	edge images built; edge pipeline running on one VM through Nuvla; test F1 passes
3	20 to 27 Oct	ingestion API, storage, retraining job, release and roll-out; tests F2 to F4
4	27 Oct to 3 Nov	full replay with and without retraining; resilience tests; report
	Tue 3 Nov 2026, 8:45	**final submission**

10 Roles and remaining actions of the teaching team

Action	Owner	Due
Distribute the data folder	M. Salimi	done, 6 Oct
Confirm the cloud quota per group (5 edge VMs + 1 cloud VM)	N. Abdennadher	9 Oct
Nuvla organisations, accounts and registry access	K. Skaburskas, John	9 Oct
Build and publish the model kit (models, features, train, verify, test vectors, μ_m)	M. Salimi	done, 6 Oct
Dry run of the full loop; confirm the flag rule, N, the fallback period and that releases occur	M. Salimi	done, 6 Oct
Confirm with Groupe E that station and node names may be shown	M. Salimi	9 Oct
Confirm the remaining 10 % of the assessment	N. Abdennadher	9 Oct

Contacts. Nabil Abdennadher (course and project), Konstantin Skaburskas and John (Nuvla platform), Mohsen Salimi (data, model kit, project follow-up).

A Payload examples

A.1 MQTT reading

```
topic: meters/5c3492e5/readings
{
  "device_id": "5c3492e5",
  "timestamp": "2026-04-01T12:15:00",
  "timestamp_utc": "2026-04-01T10:15:00Z",
  "ap_kw": -2.41,
  "ap_kw_origin": 0
}
```

A.2 Low-performance batch

```
POST /v1/lowperf
{
  "batch_id": "6f1c2a9e-3b7d-4c55-9a0e-1d2f3a4b5c6d",
  "device_id": "5c3492e5",
  "category": "pros",
  "model_version": 1,
  "day": "2026-04-12",
  "mae_kw": 0.84,
  "mae_persistence_kw": 0.61,
  "forecast_kw": [96 values],
  "actual_kw":   [96 values, null if missing],
  "actual_origin": [96 codes],
  "history_kw":  [672 values: days D-7 to D-1, null if missing],
  "history_origin": [672 codes],
  "sent_at": "2026-10-20T14:03:11Z"
}
```

`mae_kw` and `mae_persistence_kw` are the two errors of the low-performance rule; they are for monitoring, and `train.py` does not need them. All other fields are required.

A.3 Edge application (Docker Compose sketch)

The image `registry.example/p4-edge:1.0` stands for the group's own edge image (Section 5.3).

```
services:
  replayer:
    image: registry.example/p4-edge:1.0
    command: ["replayer"]
    environment: [DEVICE_IDS, REPLAY_START, REPLAY_SPEED]
    volumes: ["./data:/data:ro"]
  forecaster:
    image: registry.example/p4-edge:1.0
    command: ["forecaster"]
    environment: [DEVICE_IDS, API_URL, API_TOKEN,
                  MODEL_VERSION_CONS, MODEL_VERSION_PROS]
    volumes: ["history:/state"]
    restart: unless-stopped
  monitor:
    image: registry.example/p4-edge:1.0
    command: ["monitor"]
    environment: [DEVICE_IDS, API_URL, API_TOKEN]
    volumes: ["history:/state", "outbox:/outbox"]
    restart: unless-stopped
```

```
volumes:
  history: {}
  outbox: {}
```

A.4 Reading the metadata of a meter file

```
import json, pyarrow.parquet as pq

meta = pq.read_schema("meters/meter_27afc1ed.parquet").metadata
info = {k.decode(): v.decode() for k, v in meta.items() if k != b"pandas"}
print(info["meter_id"], info["meter_class"], info["node_id"])
codes = json.loads(info["origin_enum"])
```